

In [12]:

```
using LinearAlgebra
using Statistics
using Distributions
using Test
using Plots
using Random
using Measures
using BenchmarkTools

"""
Computes the James–Stein Eigenvector (JSE) estimator for the leading eigenvector
of a factor-based covariance matrix estimated from the high-dimensional limit.

Params:
* S: sample covariance matrix
* n: num of obs used to compute S
* use_symmetric: Symmetric wrapper for better performance

Return:
* normalized JSE estimator of the leading eigenvector
"""
function jse_estimator(S::Matrix{T}, n::Int; use_symmetric=true) where T <:
    p = size(S, 1)

    F = use_symmetric ? eigen(Symmetric(S)) : eigen(S)

    #Note to self: largest eigenvalue is last in Julia
    λ² = F.values[end]
    h = F.vectors[:, end]

    if sum(h) < 0
        h = -h
    end

    # Eq 9
    v² = (tr(S) - λ²) / (p * (n - 1))

    # Eq 8
    λ = sqrt(λ²)
    m_h = mean(h)
    s² = sum((λ*h .- λ*m_h).^2)/p

    eps_threshold = eps(T) * λ²
    if s² <= eps_threshold
        return h ./ norm(h) # 0 shrinkage needed
    end

    # Eq 7
    c_JSE = clamp(1 - v²/s², zero(T), one(T))

    # Eq 6
    h_JSE = m_h .+ c_JSE.*(h .- m_h)
    return h_JSE ./ norm(h_JSE)
end
```

```

"""
Factor based covariance matrix estimator using either raw or JSE eigenvector

Params:
* S: sample covariance matrix
* n: num of obs used to compute S
* method: :raw, :jse, or :pca for different eigenvector estimators

Return:
- estimated covariance matrix
"""
function factor_cov_matrix(S::Matrix{T}, n::Int; method=:jse) where T <: AbstractMatrix{<: Real}
    p = size(S, 1)
    F = eigen(Symmetric(S))
     $\lambda^2$  = F.values[end]
    h = F.vectors[:, end]

    # Eq. 31
     $\ell^2$  = (tr(S) -  $\lambda^2$ ) / (n - 1)

    if method == :raw
        b = h
    elseif method == :jse
        b = jse_estimator(S, n)
    elseif method == :pca
        # modified PCA: correct eigenvalue but keep sample eigenvector
        b = h
    else
        error("Unknown method: $method")
    end

    # factor covariance matrix Eq. 37
     $\Sigma_{\text{hat}}$  = ( $\lambda^2$  -  $\ell^2$ ) * (b * b') + (n/p) *  $\ell^2$  * I

    return  $\Sigma_{\text{hat}}$ 
end

"""
Simulate data from a one factor model where  $\beta \sim N(1, 0.25)$ 
factor returns with  $\sigma=0.16$ , specific returns with  $\delta=0.60$ 

Params:
* n: Num of obs
* p: Numb of vars
*  $\sigma$ : SD of factor returns (default: 0.16)
*  $\delta$ : SD of specific returns (default: 0.60)
* mean_beta: mean of the beta distribution (default: 1.0)
* var_beta: var of the beta distribution (default: 0.25)

Return:
* X: synthetic p x n data matrix
*  $\beta$ : true factor loadings
*  $\Sigma$ : true covariance matrix
"""
function simulate_factor_data(n::Int, p::Int;  $\sigma=0.16$ ,  $\delta=0.60$ , mean_beta=1.0,

```

```

    if seed != nothing
        Random.seed!(seed)
    end

    β = rand(Normal(mean_beta, sqrt(var_beta)), p)

    # (common factor f)
    f = rand(Normal(0, σ), n)

    # specific returns (with t-distribution with 5)
    ε = rand(TDist(5), (p, n)) .* (δ / sqrt(5/(5-2)))

    X = β * f' + ε

    # Eq. 11
    Σ = (σ^2) * (β * β') + (δ^2) * I

    return X, β, Σ
end

"""
Computes the minimum variance portfolio weights.

Param:
* Σ: The covariance matrix

Return:
* vector of portfolio weights that minimize variance
"""
function min_variance_portfolio(Σ::Matrix{T}) where T <: AbstractFloat
    p = size(Σ, 1)
    ones_vec = ones(p)

    # Eq. 25
    # subject to w'1 = 1
    Σ_inv = inv(Σ)
    w = (Σ_inv * ones_vec) ./ (ones_vec' * Σ_inv * ones_vec)

    return w
end

"""
The angular distance (smallest angle) between two vectors

Params:
* v1, v2: The vectors to compare

Return:
* The angle between the vectors in radians
"""
function angular_distance(v1::Vector{T}, v2::Vector{T}) where T <: AbstractF
    v1_norm = v1 ./ norm(v1)
    v2_norm = v2 ./ norm(v2)

    if sum(v1_norm) < 0
        v1_norm = -v1_norm
    end

```

```

    end
    if sum(v2_norm) < 0
        v2_norm = -v2_norm
    end

    cos_sim = clamp(dot(v1_norm, v2_norm), -1.0, 1.0)

    return acos(cos_sim)
end

====
Compute the optimization bias metric, which measures how far the estimated
portfolio weights are from the true optimal weights.

Params:
* w_est: The estimated portfolio weights
* w_true: The true optimal portfolio weights
*  $\Sigma$ : The true covariance matrix

Return:
* The optimization bias metric
====
function optimization_bias(w_est::Vector{T}, w_true::Vector{T},  $\Sigma$ ::Matrix{T})
    # Eq. 27
    tvr = (w_true' *  $\Sigma$  * w_true) / (w_est' *  $\Sigma$  * w_est)

    # 1 - TVR so that smaller is better
    return 1.0 - tvr
end

====
Compute the variance forecast ratio which measures the accuracy of the variance
forecast.

Params:
* w: portfolio weights
*  $\Sigma_{est}$ : estimated covariance matrix
*  $\Sigma_{true}$ : true covariance matrix

Return:
* variance forecast ratio (1.0 is perfect, closer the better)
====
function variance_forecast_ratio(w::Vector{T},  $\Sigma_{est}$ ::Matrix{T},  $\Sigma_{true}$ ::Matrix{T})
    # Eq. 26
    vfr = (w' *  $\Sigma_{est}$  * w) / (w' *  $\Sigma_{true}$  * w)

    return vfr
end

====
Tracking error between the estimated and true optimal portfolios.

Params:
* w_est: estimated portfolio weights
* w_true: true optimal portfolio weights
*  $\Sigma$ : true covariance matrix

```

```

Return:
* The tracking error
"""

function tracking_error(w_est::Vector{T}, w_true::Vector{T}, Σ::Matrix{T}) w
    # Eq. 28
    te_squared = (w_est - w_true)' * Σ * (w_est - w_true)

    return sqrt(te_squared)
end

"""
HL trials to compare different estimators.

Params:
* n: num of obs
* p: num of vars
* num_trials: num of sim trials
* kwargs: params passed to simulate_factor_data

Return:
* A dict with preformance metrics
"""
function run_hl_trials(n=40, p=500, num_trials=100; kwargs...)
    metrics = Dict{
        :angular => Dict{:row => Float64[], :pca => Float64[], :jse => Float64[]},
        :optim_bias => Dict{:row => Float64[], :pca => Float64[], :jse => Float64[]},
        :vfr => Dict{:row => Float64[], :pca => Float64[], :jse => Float64[]},
        :tracking_error => Dict{:row => Float64[], :pca => Float64[], :jse => Float64[]}
    }

    for trial in 1:num_trials
        X, β, Σ = simulate_factor_data(n, p, seed=trial; kwargs...)
        S = cov(X, dims=2)
        w_true = min_variance_portfolio(Σ)

        # Leading eigenvector from sample covariance matrix
        h_raw = eigen(Symmetric(S)).vectors[:, end]
        if sum(h_raw) < 0
            h_raw = -h_raw
        end

        h_jse = jse_estimator(S, n)

        # normalized true β
        β_norm = β ./ norm(β)
        if sum(β_norm) < 0
            β_norm = -β_norm
        end

        push!(metrics[:angular][:row], angular_distance(h_raw, β_norm))
        push!(metrics[:angular][:pca], angular_distance(h_raw, β_norm))
        push!(metrics[:angular][:jse], angular_distance(h_jse, β_norm))

        for method in [:row, :pca, :jse]
            Σ_est = factor_cov_matrix(S, n, method=method)
            w_est = min_variance_portfolio(Σ_est)
        end
    end
end

```

```

        push!(metrics[:optim_bias][method], optimization_bias(w_est, w_t))
        push!(metrics[:vfr][method], variance_forecast_ratio(w_est,  $\Sigma_{est}$ , w_t))
        push!(metrics[:tracking_error][method], tracking_error(w_est, w_t))
    end
end

return metrics
end

"""
Plots comparing the performance of different estimators for a given metric.

Params:
* metrics: The metrics dict from run_hl_trials
* metric_key: The key of the metric to plot
* title: you know what
"""
function plot_metrics(metrics::Dict, metric_key::Symbol, title::String="")
    boxplot(
        [metrics[metric_key][:raw], metrics[metric_key][:pca], metrics[metric_key][:jse]],
        labels=["Raw" "PCA" "JSE"],
        title=isempty(title) ? string(metric_key) : title,
        ylabel=string(metric_key),
        yformatter = :auto
    )
end

"""
Plots
Params:
* metrics: The metrics dict from run_hl_trials

Return:
* A plot object
"""
function plot_hl_results(metrics::Dict)
    p = plot(layout=(2,2), size=(1200, 800), margin=5mm)

    subplot_titles = [
        "Angular Distance (radians)",
        "Optimization Bias (smaller is better)",
        "Variance Forecast Ratio (closer to 1 is better)",
        "Tracking Error (smaller is better)"
    ]

    metric_keys = [:angular, :optim_bias, :vfr, :tracking_error]

    for (i, (key, title)) in enumerate(zip(metric_keys, subplot_titles))
        if haskey(metrics, key) &&
            all(haskey(metrics[key], method) && !isempty(metrics[key][method])
                for method in [:raw, :pca, :jse])
            boxplot!(
                p[i],
                [metrics[key][:raw], metrics[key][:pca], metrics[key][:jse]]
            )
        end
    end
end

```

```

        labels=["Raw" "PCA" "JSE"],
        title=title,
        titlefontsize=10,
        xtickfontsize=8,
        ytickfontsize=8,
        yformatter=:auto
    )
else
    annotate!(p[i], 0.5, 0.5, text("No data available for $(key)", 1
end
end

return p
end

"""
Benchmark different implementations of JSE estimator.

Params:
* n: num of obs
* p: num of vars
"""
function benchmark_implementations(n::Int, p::Int)
    X,  $\beta$ ,  $\Sigma$  = simulate_factor_data(n, p)
    S = cov(X, dims=2)

    println("Benchmarking jse_estimator with Symmetric matrix:")
    @btime jse_estimator($S, $n, use_symmetric=true)

    println("\nBenchmarking jse_estimator without Symmetric matrix:")
    @btime jse_estimator($S, $n, use_symmetric=false)

    println("\nBenchmarking factor_cov_matrix with raw method:")
    @btime factor_cov_matrix($S, $n, method=:raw)

    println("\nBenchmarking factor_cov_matrix with jse method:")
    @btime factor_cov_matrix($S, $n, method=:jse)
end

function run_hl_trials_verbose(n=40, p=500, num_trials=5; kwargs...)
    metrics = run_hl_trials(n, p, num_trials; kwargs...)

    println("=== Mean Performance Metrics ($num_trials trials) ===")
    println("Angular Distance (radians, smaller is better):")
    println("  Raw: $(round(mean(metrics[:angular][:raw]), digits=4))")
    println("  PCA: $(round(mean(metrics[:angular][:pca]), digits=4))")
    println("  JSE: $(round(mean(metrics[:angular][:jse]), digits=4))")
    println("  Improvement: $(round(mean(metrics[:angular][:raw]) - mean(met

    println("\nOptimization Bias (smaller is better):")
    println("  Raw: $(round(mean(metrics[:optim_bias][:raw]), digits=4))")
    println("  PCA: $(round(mean(metrics[:optim_bias][:pca]), digits=4))")
    println("  JSE: $(round(mean(metrics[:optim_bias][:jse]), digits=4))")
    println("  Raw-JSE Improvement: $(round(mean(metrics[:optim_bias][:raw])")
    println("  PCA-JSE Improvement: $(round(mean(metrics[:optim_bias][:pca])

```

```

println("\nVariance Forecast Ratio (closer to 1 is better):")
println("  Raw: $(round(mean(metrics[:vfr][:raw]), digits=4))")
println("  PCA: $(round(mean(metrics[:vfr][:pca]), digits=4))")
println("  JSE: $(round(mean(metrics[:vfr][:jse]), digits=4))")

println("\nTracking Error (smaller is better):")
println("  Raw: $(round(mean(metrics[:tracking_error][:raw]), digits=4))")
println("  PCA: $(round(mean(metrics[:tracking_error][:pca]), digits=4))")
println("  JSE: $(round(mean(metrics[:tracking_error][:jse]), digits=4))")

    return metrics
end

@testset "JSE HL Performance" begin
    # small test
    metrics = run_hl_trials(40, 100, 10)

    # angular improvement
    @test mean(metrics[:angular][:jse]) < mean(metrics[:angular][:raw])

    # optimization bias with tolerance
    @test mean(metrics[:optim_bias][:jse]) < mean(metrics[:optim_bias][:raw])

    # variance forecast ratio (should be closer to 1.0)
    avg_raw = mean(metrics[:vfr][:raw])
    avg_jse = mean(metrics[:vfr][:jse])

    # distance from 1.0
    dist_raw = abs(avg_raw - 1.0)
    dist_jse = abs(avg_jse - 1.0)

    @test dist_jse < dist_raw

    # tracking error should be smaller for JSE
    @test mean(metrics[:tracking_error][:jse]) < mean(metrics[:tracking_error][:raw])
end

function run_paper_simulation(plot_results=true; n=40, p=500, trials=100)
    println("simulations with (n=$n, p=$p, $trials trials)...")
    metrics = run_hl_trials_verbose(n, p, trials,  $\sigma=0.16$ ,  $\delta=0.60$ , mean_beta=0.5)

    if plot_results
        plt = plot_hl_results(metrics)
        display(plt)
    end

    return metrics
end

Random.seed!(42)
metrics = run_paper_simulation(true, n=40, p=500, trials=100)

benchmark_implementations(40, 500)

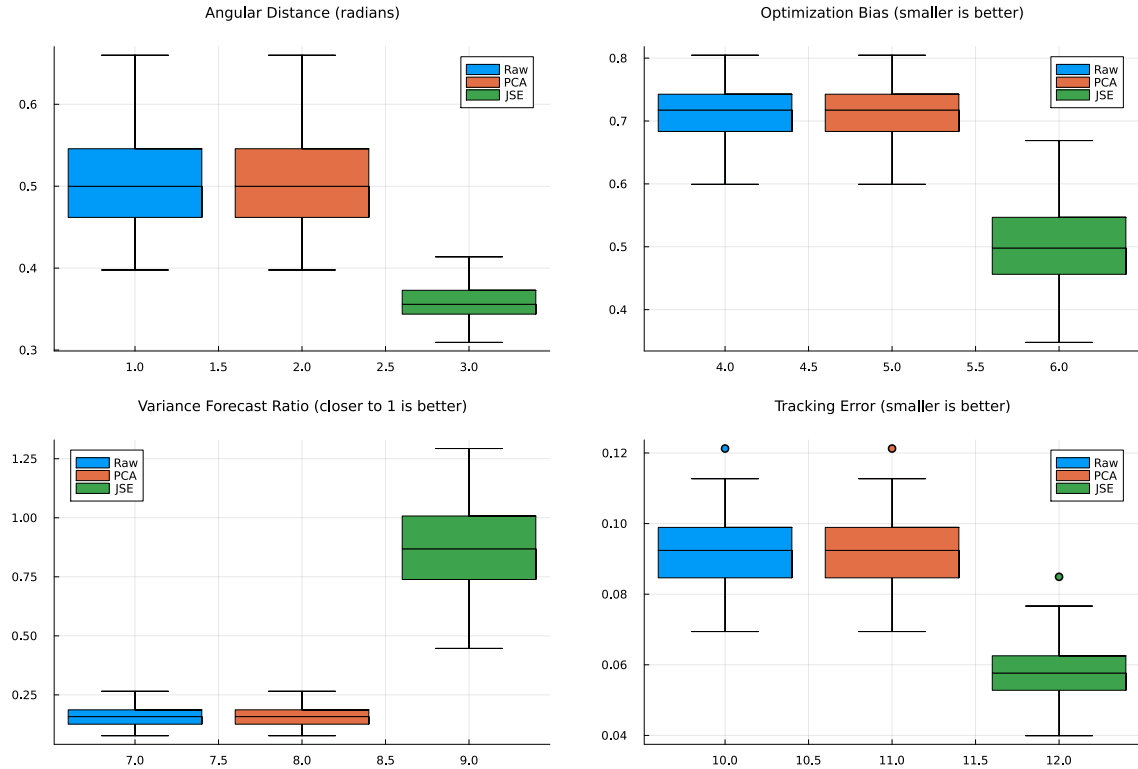
```

Test Summary: | **Pass** **Total** **Time**
JSE HL Performance | 4 4 0.6s
simulations with (n=40, p=500, 100 trials)...
=== Mean Performance Metrics (100 trials) ===
Angular Distance (radians, smaller is better):
Raw: 0.5072
PCA: 0.5072
JSE: 0.3579
Improvement: 0.1493

Optimization Bias (smaller is better):
Raw: 0.7138
PCA: 0.7138
JSE: 0.5002
Raw-JSE Improvement: 0.2136
PCA-JSE Improvement: 0.2136

Variance Forecast Ratio (closer to 1 is better):
Raw: 0.1584
PCA: 0.1584
JSE: 0.8635

Tracking Error (smaller is better):
Raw: 0.0921
PCA: 0.0921
JSE: 0.0583



Benchmarking jse_estimator with Symmetric matrix:

36.471 ms (41 allocations: 5.93 MiB)

Benchmarking jse_estimator without Symmetric matrix:

36.682 ms (41 allocations: 5.93 MiB)

Benchmarking factor_cov_matrix with raw method:

37.690 ms (23 allocations: 11.63 MiB)

Benchmarking factor_cov_matrix with jse method:

74.267 ms (64 allocations: 17.55 MiB)

Out[12]: 500×500 Matrix{Float64}:

0.397075	0.0127939	0.0377634	...	0.0428592	0.0324763	0.0330304
0.0127939	0.35467	0.0105107		0.0119291	0.00903917	0.0091934
0.0377634	0.0105107	0.382133		0.0352106	0.0266806	0.0271358
0.0273449	0.00761093	0.0224649		0.0254964	0.0193197	0.0196494
0.0207443	0.0057738	0.0170423		0.019342	0.0146563	0.0149064
0.0247261	0.00688204	0.0203135	...	0.0230546	0.0174695	0.0177675
0.0337886	0.00940442	0.0277587		0.0315045	0.0238723	0.0242797
0.0309798	0.00862266	0.0254512		0.0288856	0.0218879	0.0222614
0.0443263	0.0123374	0.0364159		0.0413299	0.0313174	0.0318518
0.0465322	0.0129514	0.0382281		0.0433866	0.032876	0.0334369
0.0310819	0.00865108	0.0255351	...	0.0289808	0.02196	0.0223347
0.0332168	0.00924528	0.027289		0.0309714	0.0234684	0.0238688
0.0244057	0.00679289	0.0200503		0.0227559	0.0172432	0.0175374
:			⋱			
0.035834	0.00997373	0.0294391		0.0334116	0.0253175	0.0257495
0.0297864	0.0082905	0.0244708		0.0277729	0.0210447	0.0214038
0.0259392	0.0072197	0.0213101	...	0.0241857	0.0183266	0.0186393
0.0280586	0.00780959	0.0230513		0.0261618	0.019824	0.0201622
0.0403492	0.0112304	0.0331485		0.0376216	0.0285076	0.028994
0.0309799	0.00862268	0.0254513		0.0288857	0.0218879	0.0222614
0.0322493	0.008976	0.0264942		0.0300693	0.0227848	0.0231736
0.0351104	0.00977233	0.0288447	...	0.032737	0.0248062	0.0252295
0.0370844	0.0103217	0.0304663		0.0345775	0.0262009	0.0266479
0.0428592	0.0119291	0.0352106		0.391071	0.0302809	0.0307976
0.0324763	0.00903917	0.0266806		0.0302809	0.374054	0.0233367
0.0330304	0.0091934	0.0271358		0.0307976	0.0233367	0.374844